

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Комп'ютерна графіка та обробка зображень»

Робота №4

Тема «Інтерполяція, індекси малювання та проєкції»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Туленков А.В.

2025

ЗАВДАННЯ

Завдання на роботу зі сторінки 21:

1. Змініть фігуру та її колір.
2. Запишіть коротке відео результату.
3. Наведіть коментарі до коду.
4. Внесіть індивідуальні зміни до коду.

РЕЗУЛЬТАТ

```
#include <stdio.h>
#include <string.h>
#include <cmath>
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>

// Ширина та висота вікна
const GLint WIDTH = 800, HEIGHT = 800;
// Конвертація градусів у радіани
const float toRadians = 3.14159265f / 180.0f;

// Змінні для анімації
float curAngle = 0.0f; // Поточний кут обертання
GLuint VBO, VAO, IBO, shader, uniformModel, uniformProjection;
bool direction = true; // Напрямок руху (для зміщення)
float triOffset = 0.0f; // Поточне зміщення по осі X
float triMaxOffset = 0.5f; // Максимальне зміщення по осі X
float triIncrement = 0.0005f; // Крок зміщення
bool sizeDirection = true; // Напрямок зміни розміру (для масштабування)
float curSize = 0.4f; // Поточний коефіцієнт масштабування
float maxSize = 0.8f; // Максимальний коефіцієнт масштабування
float minSize = 0.1f; // Мінімальний коефіцієнт масштабування

// Код вершинного шейдера
static const char *vShader = "
\n #version 330
\n
\n layout(location = 0) in vec3 pos;
\n
\n out vec4 vCol;
\n
\n uniform mat4 model;
\n"
```

```

uniform mat4 projection;
\n\
void main()
\n
{
    \n\
    gl_Position = projection * model * vec4(pos, 1.0);
    \n\
    // Фіксований червоний колір для куба
    vCol = vec4(1.0f, 0.0f, 0.0f, 1.0f);
    \n
}
";

// Код фрагментного шейдера
static const char *fShader = "
\n #version 330
\n\
in vec4 vCol;
\n\
out vec4 colour;
\n\
void main()
\n
{
    \n\
    colour = vCol;
    \n
}
";

// Функція для створення куба (використовуємо 12 трикутників)
void CreateCube()
{
    // 36 індексів для 12 трикутників, які формують 6 граней куба
    unsigned int indices[] = {
        // Передня грань
        0, 1, 2, 2, 3, 0,
        // Права грань
        1, 5, 6, 6, 2, 1,
        // Задня грань
        7, 6, 5, 5, 4, 7,
        // Ліва грань
        4, 0, 3, 3, 7, 4,
        // Верхня грань
        3, 2, 6, 6, 7, 3,
        // Нижня грань
        4, 5, 1, 1, 0, 4
    };

    // 8 вершин куба (кожна координата від -1.0f до 1.0f)
    GLfloat vertices[] = {
        // x      y      z
        -1.0f, -1.0f,  1.0f, // 0
        1.0f, -1.0f,  1.0f, // 1
        1.0f,  1.0f,  1.0f, // 2
        -1.0f,  1.0f,  1.0f, // 3
        -1.0f, -1.0f, -1.0f, // 4

```

```

        1.0f, -1.0f, -1.0f, // 5
        1.0f,  1.0f, -1.0f, // 6
        -1.0f,  1.0f, -1.0f // 7
    };

    // Створення та прив'язка VAO (Vertex Array Object)
    glGenVertexArrays(1, &VAO);
    glBindVertexArray(VAO);

    // Створення та прив'язка IBO (Index Buffer Object)
    glGenBuffers(1, &IBO);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, IBO);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices,
GL_STATIC_DRAW);

    // Створення та прив'язка VBO (Vertex Buffer Object)
    glGenBuffers(1, &VBO);
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
GL_STATIC_DRAW);

    // Вказівник на дані вершин (layout(location = 0) in vec3 pos)
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);
    glEnableVertexAttribArray(0);

    // Відв'язка буферів
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0);
    glBindVertexArray(0);
}

// Функція для додавання шейдера до програми
void AddShader(GLuint theProgram, const char *shaderCode, GLenum
shaderType)
{
    GLuint theShader = glCreateShader(shaderType);
    const GLchar *theCode[1];
    theCode[0] = shaderCode;
    GLint codeLength[1];
    codeLength[0] = strlen(shaderCode);

    glShaderSource(theShader, 1, theCode, codeLength);
    glCompileShader(theShader);

    GLint result = 0;
    GLchar eLog[1024] = {0};

    // Перевірка успішності компіляції шейдера
    glGetShaderiv(theShader, GL_COMPILE_STATUS, &result);
    if (!result)
    {
        glGetShaderInfoLog(theShader, 1024, NULL, eLog);
        fprintf(stderr, "Error compiling the %d shader: '%s'\n",
shaderType, eLog);
        return;
    }

    glAttachShader(theProgram, theShader);

```

```

}

// Функція для компіляції та лінування шейдерів
void CompileShaders()
{
    shader = glCreateProgram();

    if (!shader)
    {
        printf("Failed to create shader\n");
        return;
    }

    AddShader(shader, vShader, GL_VERTEX_SHADER); // Додавання
вершинного шейдера
    AddShader(shader, fShader, GL_FRAGMENT_SHADER); // Додавання
фрагментного шейдера

    GLint result = 0;
    GLchar eLog[1024] = {0};

    glLinkProgram(shader); // Лінування програми шейдерів

    // Перевірка успішності лінування
    glGetProgramiv(shader, GL_LINK_STATUS, &result);
    if (!result)
    {
        glGetProgramInfoLog(shader, sizeof(eLog), NULL, eLog);
        printf("Error linking program: '%s'\n", eLog);
        return;
    }

    glValidateProgram(shader); // Валідація програми

    // Перевірка успішності валідації
    glGetProgramiv(shader, GL_VALIDATE_STATUS, &result);
    if (!result)
    {
        glGetProgramInfoLog(shader, sizeof(eLog), NULL, eLog);
        printf("Error validating program: '%s'\n", eLog);
        return;
    }

    // Отримання location uniform-змінних
    uniformModel = glGetUniformLocation(shader, "model");
    uniformProjection = glGetUniformLocation(shader, "projection");
}

int main()
{
    // Ініціалізація GLFW
    if (!glfwInit())
    {
        printf("GLFW failed to start :(");
        glfwTerminate();
        return 1;
    }
}

```

```

// Налаштування версії OpenGL (Core Profile 3.3)
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);

// Створення вікна
GLFWwindow *mainWindow = glfwCreateWindow(WIDTH, HEIGHT, "Test
Window", NULL, NULL);
if (!mainWindow)
{
    printf("GLFW window creation failed :(");
    glfwTerminate();
    return 1;
}

// Отримання розмірів буфера кадру
int bufferWidth, bufferHeight;
glfwGetFramebufferSize(mainWindow, &bufferWidth, &bufferHeight);

// Встановлення поточного контексту
glfwMakeContextCurrent(mainWindow);

// Дозволити сучасні функції розширення
glewExperimental = GL_TRUE;

// Ініціалізація GLEW
if (glewInit() != GLEW_OK)
{
    printf("GLEW failed to start >_<");
    glfwDestroyWindow(mainWindow);
    glfwTerminate();
    return 1;
}

// Включення тесту глибини
glEnable(GL_DEPTH_TEST);

// Встановлення viewport
glViewport(0, 0, bufferWidth, bufferHeight);

// Створення фігури (Куба)
CreateCube();

// Компіляція шейдерів
CompileShaders();

// Створення матриці проекції (Перспективна проекція)
glm::mat4 projection = glm::perspective(glm::radians(45.0f),
(GLfloat)bufferWidth / (GLfloat)bufferHeight, 0.1f, 100.0f);

// Основний цикл програми
while (!glfwWindowShouldClose(mainWindow))
{
    // Обробка подій (наприклад, натискання клавіш)
    glfwPollEvents();

    // Логіка анімації: Зміщення куба по осі X

```

```

    if (direction)
    {
        triOffset += triIncrement;
    }
    else
    {
        triOffset -= triIncrement;
    }

    if (abs(triOffset) >= triMaxOffset)
    {
        direction = !direction; // Зміна напрямку руху
    }

    // Логіка анімації: Обертання куба
    curAngle += 0.2f; // Збільшено швидкість обертання
    if (curAngle >= 360)
    {
        curAngle -= 360;
    }

    // Логіка анімації: Масштабування куба
    if (sizeDirection)
    {
        curSize += 0.001f; // Збільшено швидкість масштабування
    }
    else
    {
        curSize -= 0.001f;
    }

    if (curSize >= maxSize || curSize <= minSize)
    {
        sizeDirection = !sizeDirection; // Зміна напрямку
масштабування
    }

    // Очищення буфера кольору (Темно-синій колір)
    glClearColor(0.0f, 0.0f, 0.2f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glUseProgram(shader);

    // Створення матриці моделі (Model Matrix)
    glm::mat4 model(1.0f);

    // 1. Трансляція (переміщення) куба. Рух по осі X.
    model = glm::translate(model, glm::vec3(triOffset, 0.0f,
-5.0f)); // Зміщення по X, z = -5.0f (для віддалення від камери)

    // 2. Обертання куба навколо осі Z
    model = glm::rotate(model, curAngle * toRadians,
glm::vec3(0.0f, 0.0f, 1.0f));

    // 3. Масштабування куба
    model = glm::scale(model, glm::vec3(curSize, curSize,
curSize));

```

```

        // Передача uniform-змінних у шейдер
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformProjection, 1, GL_FALSE,
glm::value_ptr(projection));

        // Рендеринг куба
        glBindVertexArray(VAO);
        glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, IBO);
        glDrawElements(GL_TRIANGLES, 36, GL_UNSIGNED_INT, 0); // 36
індексів для куба
        glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0);
        glBindVertexArray(0);

        glUseProgram(0);

        // Обмін буферів (показ відмальованого кадру)
        glfwSwapBuffers(mainWindow);
    }

    // Завершення роботи GLFW
    return 0;
}

```